BÀI 3. HEURISTIC – NGUYÊN LÝ THỨ TỰ VÀ A* SEARCH

1. Mục tiêu

- Hiểu nguyên lý thứ tự trong Best-First Search: frontier (OPEN) được sắp theo một hàm ưu tiên và node có ưu tiên tốt nhất được mở rộng trước.
- Phân biệt rõ g(n), h(n), h*(n) và f(n); giải thích được vì sao A* dùng f(n)=g(n)+h(n).
- Phân biệt heuristic admissible và consistent; kiểm tra hai tính chất này trên grid 4 hướng có step cost = 1.
- Cài đặt A* bằng priority queue, best_g và parent; hiểu relaxation và cách bỏ stale entry trong priority queue.
- So sánh Dijkstra, A*-h0, A*-Manhattan, A*-Euclidean, Weighted A* và Greedy Best-First Search trên cùng dữ liệu.
- Đánh giá thuật toán bằng path cost, optimality gap, nodes expanded, max frontier và runtime; rút ra kết luận dựa trên số liệu.

2. Lý thuyết thuật toán

2.1. Nguyên lý thứ tự trong Best-First Search

Trong các thuật toán Best-First Search, frontier (hay OPEN) chứa các node đã phát hiện nhưng chưa xử lý xong. "Nguyên lý thứ tự" chính là quy tắc dùng để chọn node nào được lấy ra trước. Thông thường frontier được cài bằng priority queue; node có giá trị ưu tiên nhỏ nhất sẽ được pop trước.

Các đại lượng dùng trong Lab:

- g(n): chi phí tốt nhất hiện biết từ start đến node n.
- h(n): heuristic – ước lượng chi phí còn lại từ n đến goal.
- h*(n): chi phí tối ưu thật từ n đến goal.
- f(n): hàm đánh giá / hàm ưu tiên dùng để sắp thứ tự các node trong frontier.

Ba chiến lược cơ bản khác nhau chủ yếu ở cách định nghĩa f(n):

$$\text{Greedy: } f(n) = h(n)$$

$$\text{Dijkstra / Uniform} - \text{Cost: } f(n) = g(n)$$

$$A*: f(n) = g(n) + h(n)$$

$$\text{Weighted } A*: f_w(n) = g(n) + w \cdot h(n),\ w > 1$$

Thuật toán	Hàm ưu tiên	Ý nghĩa
Greedy Best-First	f(n)=h(n)	Chỉ nhìn "còn bao xa tới goal"; thường định hướng mạnh nhưng không bảo đảm đường tối ưu.
Dijkstra / Uniform-Cost	f(n)=g(n)	Chỉ nhìn chi phí đã đi; tối ưu khi trọng số cạnh không âm.
A*	f(n)=g(n)+h(n)	Kết hợp chi phí đã đi và ước lượng còn lại; có thể vừa tối ưu vừa mở ít node hơn Dijkstra nếu h phù hợp.
Weighted A*	f(n)=g(n)+w·h(n)	Tăng ảnh hưởng của heuristic; thường giảm tìm kiếm nhưng có thể chấp nhận đường suboptimal.

2.2. Heuristic: admissible, consistent và các khoảng cách trên grid

Heuristic tốt cần cung cấp thông tin định hướng nhưng không làm A* đánh mất các bảo đảm cần thiết. Hai khái niệm quan trọng là admissible và consistent.

Admissible: heuristic không được ước lượng vượt quá chi phí tối ưu thật còn lại:

$$0 \le h(n) \le h*(n), \text{ và } h(\text{goal}) = 0$$

Consistent (monotone): với mọi cạnh n → n′ có chi phí c(n,n′), heuristic phải thỏa bất đẳng thức tam giác:

$$h(n) \leq c(n, n') + h(n')$$

Nếu h consistent thì h cũng admissible (trong các điều kiện chuẩn) và giá trị f dọc theo đường đi không giảm. Điều này giúp A* xử lý graph hiệu quả và tránh phải "mở lại" node đã được chốt theo cách cài đặt chuẩn.

Trong grid 4 hướng, node n=(x,y), goal=(x_G,y_G), mỗi bước ngang/dọc có cost = 1. Hai heuristic dùng trong Lab là:

h_M(n) = |x − x_G| + |y − y_G|

$$h_E(n) = \sqrt{((x - x_G)^2 + (y - y_G)^2)}$$

Manhattan là số bước tối thiểu cần thiết nếu bỏ qua vật cản, nên không thể lớn hơn chi phí đường đi thật trên grid 4 hướng. Euclidean là khoảng cách đường thẳng, cũng là lower bound. Trên cùng hai điểm ta có:

$$h_E(n) \leq h_M(n) \leq h*(n)$$

Vì Manhattan thường lớn hơn hoặc bằng Euclidean nhưng vẫn admissible, nó thường "sắc" hơn trong grid 4 hướng và do đó có thể mở ít node hơn A*-Euclidean.

Với Weighted A* dùng w=1.5:

$$f(n) = g(n) + 1.5 \cdot h_M(n)$$

Do 1.5·h_M(n) có thể lớn hơn h*(n), heuristic đã được nhân trọng số không còn admissible nói chung. Kết quả có thể nhanh hơn nhưng optimality gap có thể dương.

2.3. A* Search: dữ liệu cần lưu và cơ chế relaxation

A* không chỉ cần biết một node đã từng xuất hiện hay chưa. Một node có thể được phát hiện qua nhiều đường khác nhau; đường phát hiện sau có thể rẻ hơn đường trước. Vì vậy cài đặt đúng cần ba cấu trúc chính:

- OPEN / priority queue: lưu các entry chờ xét, sắp theo f(n).
- best_g[n]: chi phí nhỏ nhất hiện biết từ start đến n.
- parent[n]: node cha tương ứng với best_g[n], dùng để dựng lại đường đi tối ưu hiện biết.

Khi đang ở node n và xét successor s, ta tính chi phí ứng viên:

$$new_g = best_g[n] + c(n, s)$$

Nếu đường mới tốt hơn đường đã biết, thực hiện relaxation:

$$\text{nếu } new_g < best_g[s] \Rightarrow best_g[s] \leftarrow new_g,\ parent[s] \leftarrow n$$

$$f(s) \leftarrow new_g + h(s)$$

Priority queue thông thường không hỗ trợ giảm khóa trực tiếp (decrease-key). Cách đơn giản là push một entry mới. Entry cũ vẫn còn trong heap; khi pop ra, nếu g trong entry lớn hơn best_g hiện tại thì đó là stale entry và phải bỏ qua.

> **Lưu ý:** Không dùng visited đơn giản theo kiểu "node đã thấy thì khóa vĩnh viễn ngay lần đầu phát hiện". Cách này có thể làm mất một đường tốt hơn đến cùng node trong graph có trọng số.

Điều kiện dừng cũng rất quan trọng: A* chuẩn không dừng chỉ vì vừa sinh ra goal. Thuật toán dừng khi goal được pop khỏi OPEN với entry hợp lệ và có ưu tiên nhỏ nhất tại thời điểm đó.

Các bước A* Search:

1. Khởi tạo best_g[start]=0, parent[start]=None; push start vào OPEN với f=h(start).
2. Pop entry có f nhỏ nhất.
3. Nếu entry là stale (g > best_g[node]) thì bỏ qua và pop tiếp.
4. Nếu node là goal thì dựng lại path bằng parent và trả về g(goal).
5. Với mỗi successor s, tính new_g = g(node)+c(node,s).
6. Nếu new_g tốt hơn best_g[s], cập nhật best_g, parent và push entry mới với f=new_g+h(s).
7. Lặp cho tới khi tìm được goal hoặc OPEN rỗng.

```
A-STAR(graph, start, goal, h):
  OPEN <- priority queue ordered by f
  best_g[start] <- 0
  parent[start] <- None
  push (h(start), 0, start) into OPEN
```

```
while OPEN is not empty:
   (f, g, n) <- pop smallest entry

   if g > best_g[n]:
      continue              # stale entry

   if n == goal:
      return reconstruct(parent, goal), g

   for each successor s of n:
      new_g <- g + cost(n, s)

      if new_g < best_g.get(s, infinity):
         best_g[s] <- new_g
         parent[s] <- n
         new_f <- new_g + h(s)
         push (new_f, new_g, s) into OPEN

return failure
```

2.4. Quan hệ giữa Dijkstra, A*, Greedy và Weighted A*

Các thuật toán trong Lab có thể xem là cùng một khung Best-First Search nhưng khác hàm ưu tiên:

$$\text{Dijkstra} \equiv A* \text{ với } h(n) = 0 \Rightarrow f(n) = g(n)$$

$$\text{Greedy} - \text{Manhattan: } f(n) = h_M(n)$$

$$A* - \text{Manhattan: } f(n) = g(n) + h_M(n)$$

$$A* - \text{Euclidean: } f(n) = g(n) + h_E(n)$$

$$\text{Weighted } A*: \ f(n) = g(n) + 1.5 \cdot h_M(n)$$

Thuộc tính	Dijkstra	A* chuẩn	Weighted A*	Greedy
Có dùng g(n)?	Có	Có	Có	Không
Có dùng h(n)?	Không	Có	Có, trọng số >1	Có
Bảo đảm tối ưu trong Lab?	Có	Có với Manhattan/Euclidean	Không bảo đảm	Không bảo đảm
Xu hướng expanded nodes	Nhiều	Ít hơn nếu h tốt	Thường ít hơn A*	Có thể rất ít
Đánh đổi chính	Tìm rộng	Cân bằng tốt	Tốc độ ↔ optimality	Tốc độ ↔ chất lượng path

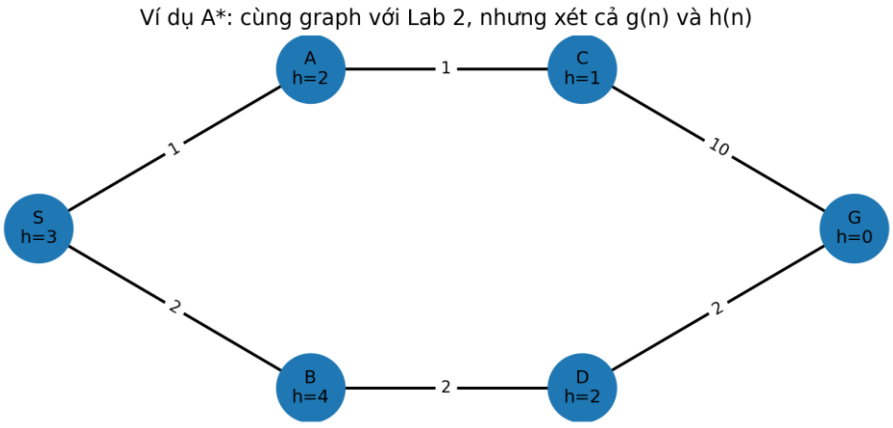
Độ phức tạp phụ thuộc không gian trạng thái và chất lượng heuristic. Với explicit graph, thao tác heap cho mỗi lần push/pop có chi phí O(log |OPEN|). Nếu mỗi cạnh được relaxation hữu hạn lần, cách nhìn gần đúng thường dùng là:

$O((|V| + |E|) \cdot \log |V|)$

Tuy nhiên, trong search tree lớn hoặc heuristic yếu, số trạng thái A* phải xét vẫn có thể tăng rất nhanh. Vì vậy trong Lab ta đo trực tiếp nodes expanded, max frontier và runtime thay vì chỉ dựa vào Big-O.

2.5. Ví dụ minh họa A*

Dùng lại graph có trọng số của Lab 2. Start=S, Goal=G. Số trên cạnh là chi phí thật; h được ghi trong từng node. Mục tiêu là thấy vì sao A* không bị "hấp dẫn" chỉ bởi h(G)=0.



Hình 1. Graph có trọng số minh họa A* Search

Bước	Node lấy ra	g	h	f=g+h	Frontier / cập nhật chính
1	S	0	3	3	Sinh A: g=1,f=3; B: g=2,f=6
2	A	1	2	3	Sinh C: g=2,f=3; B vẫn f=6
3	C	2	1	3	Sinh G: g=12,f=12. Vì B có f=6 nên chưa chọn G.
4	B	2	4	6	Sinh D: g=4,f=6
5	D	4	2	6	Tìm đường tốt hơn đến G: best_g[G] giảm từ 12 xuống 6.
6	G	6	0	6	Goal được pop với cost 6 → dừng.

Tính chi phí hai nhánh:

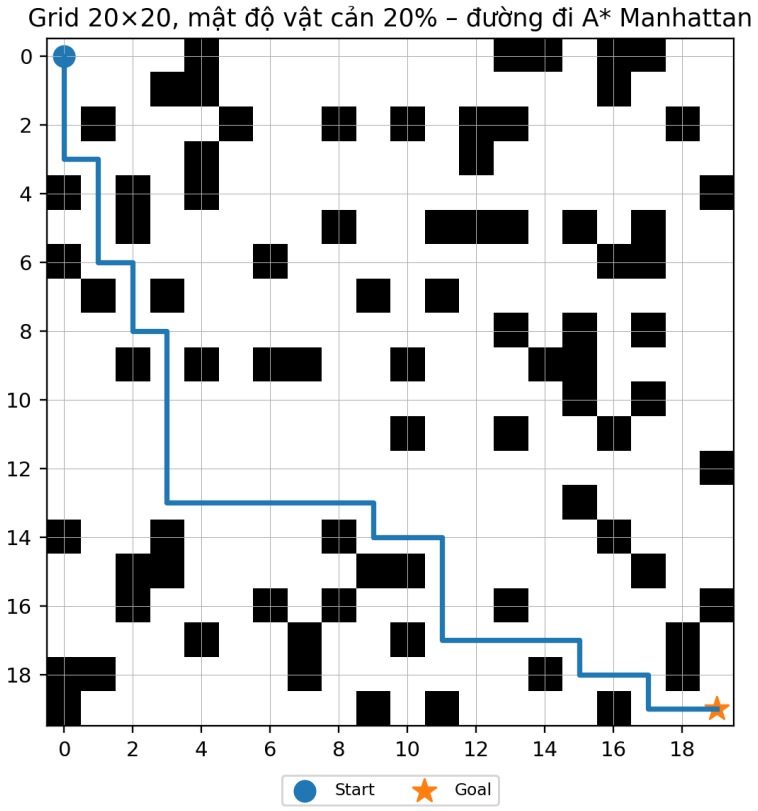
$$S \rightarrow A \rightarrow C \rightarrow G:\ 1 + 1 + 10 = 12$$

$$S \rightarrow B \rightarrow D \rightarrow G:\ 2 + 2 + 2 = 6$$

Ở bước 3, A* đã phát hiện G nhưng f(G)=12, trong khi B còn f(B)=6. Vì vậy thuật toán tiếp tục mở B, sau đó D tìm được đường rẻ hơn đến G và cập nhật best_g[G] từ 12 xuống 6. Kết quả cuối cùng là đường tối ưu S → B → D → G với cost = 6.

3. Thực nghiệm và tiêu chí đánh giá

Thực nghiệm dùng grid 20×20, start=(0,0), goal=(19,19), chuyển động 4 hướng và mỗi bước có cost = 1. Sinh ba mức mật độ vật cản 10%, 20%, 30%. Mỗi mức sử dụng cùng 10 seed cố định; chỉ giữ các grid có đường nối từ start đến goal để tất cả thuật toán được so sánh trên cùng bài toán hợp lệ.



Hình 2. Ví dụ grid 20×20 có vật cản và đường đi A*-Manhattan

Biến thí nghiệm	**Giá trị**
Kích thước grid	20×20
Start / Goal	(0,0) / (19,19)
Mật độ vật cản	10%, 20%, 30%
Seed	10 seed cố định cho mỗi mật độ
Chuyển động	4 hướng; step cost = 1
Baseline tối ưu	Dijkstra
A* heuristic	h=0; Manhattan; Euclidean; 1.5×Manhattan
Baseline định hướng	Greedy-Manhattan
Runtime benchmark	Lặp 5 lần cho mỗi cấu hình trên cùng grid; lấy thời gian trung bình

3.1. Các chỉ số đánh giá

Path cost của một đường P=(v₀,v₁,…,v_k) là tổng chi phí các cạnh trên đường:

$$\text{Cost}(P) = \Sigma_{i=0}\text{^}\{k-1\}\, c(v_i, v_{i+1})$$

Trong grid unit-cost của Lab, mỗi cạnh có cost = 1 nên:

Cost(P) = số bước = |P| − 1

Dijkstra cung cấp cost tối ưu cho từng grid, ký hiệu cost_opt. Optimality gap của thuật toán A trên seed i là:

$$\text{Gap}_i(\%) = ((\text{cost}_i - \text{cost_opt},_i) / \text{cost_opt},_i) \times 100$$

Gap = 0% nghĩa là thuật toán tìm được đường có cùng cost với Dijkstra; Gap > 0% cho biết mức đánh đổi về chất lượng đường đi.

Các giá trị báo cáo trong bảng 5.1 là trung bình trên N=10 seed:

$$\text{Mean Cost} = (1/N) \cdot \Sigma_{i=1}^{N} \text{cost}_i$$
$$\text{Mean Gap} = (1/N) \cdot \Sigma_{i=1}^{N} \text{Gap}_i$$
$$\text{Mean Expanded} = (1/N) \cdot \Sigma_{i=1}^{N} \text{expanded}_i$$
$$\text{Mean Max Frontier} = (1/N) \cdot \Sigma_{i=1}^{N} \text{max_frontier}_i$$
$$\text{Mean Runtime} = (1/N) \cdot \Sigma_{i=1}^{N} \text{runtime}_i$$

Tiêu chí / chỉ số	Cách đo	Ý nghĩa
Success / Valid path	Path tồn tại, bắt đầu ở start, kết thúc ở goal, các bước kề nhau và không đi qua obstacle.	Kiểm tra tính đúng của nghiệm.
Path cost	Tổng chi phí các cạnh trên path; với unit-cost = số bước.	So với cost_opt của Dijkstra.
Optimality gap (%)	100×(cost−cost_opt)/cost_opt.	0% là tối ưu; >0% thể hiện đánh đổi chất lượng.
Nodes expanded	Số entry non-stale được pop và thực sự xử lý.	Đo lượng công việc tìm kiếm.
Max frontier	Kích thước OPEN lớn nhất trong một lần chạy.	Ước lượng nhu cầu bộ nhớ.
Runtime (ms)	Dùng perf_counter; đo riêng phần thuật toán; trung bình 5 lần.	Đo tốc độ trên cùng môi trường.
Success rate	Số grid có nghiệm / tổng số grid thử nghiệm.	Đánh giá độ ổn định nếu cho phép grid không reachable.

3.2. Nguyên tắc so sánh công bằng

- Cùng một density và seed phải tạo đúng cùng một grid cho tất cả thuật toán.
- Start, goal, quy tắc chuyển động và step cost phải giống nhau.
- Dijkstra được chạy trên chính grid đó để lấy cost_opt riêng cho seed đó.
- Không tính thời gian sinh grid, vẽ hình hoặc plt.show() vào runtime của thuật toán.
- Cùng một quy ước đếm expanded: trong Lab này, mỗi node non-stale được pop được tính là một lần expanded; goal cũng được tính khi được pop.
- Khi đo runtime, nên lặp nhiều lần và lấy trung bình để giảm nhiễu do hệ điều hành.

4. Hướng dẫn cài đặt từng bước

Sinh viên chạy từng cell theo thứ tự và kiểm tra kết quả sau mỗi bước. Không dùng NetworkX astar_path(), shortest_path() hoặc hàm tương đương để thay cho A* tự cài.

4.1. Thư viện

Mục đích: Nạp các cấu trúc dữ liệu và thư viện cần thiết cho toàn bộ thí nghiệm. Các thư viện được giữ ở mức tối thiểu để sinh grid, cài priority queue, đo thời gian, tổng hợp dữ liệu và trực quan hóa kết quả.

Giải thích: deque được dùng cho BFS kiểm tra khả năng nối start–goal; count() tạo số thứ tự tăng dần để phá hòa khi hai phần tử trong heap có cùng mức ưu tiên; perf_counter() đo runtime với độ phân giải cao; heapq cài priority queue; math cung cấp căn bậc hai cho Euclidean; random sinh obstacle theo seed; pandas tổng hợp bảng kết quả; matplotlib.pyplot vẽ grid và path.

```
from collections import deque
from itertools import count
from time import perf_counter
import heapq
import math
import random
import pandas as pd
import matplotlib.pyplot as plt
```

4.2. Dựng lại đường đi từ parent

Mục đích: Hàm reconstruct_path(parent, goal) dựng lại toàn bộ đường đi sau khi thuật toán đã tới goal. Trong lúc tìm kiếm, mỗi node chỉ lưu node cha tốt nhất thay vì lưu cả path, nhờ đó tiết kiệm bộ nhớ.

Tham số và kết quả: parent là dictionary dạng parent[child] = parent_node; goal là đích cần truy vết. Nếu goal chưa có trong parent, hàm trả về None. Nếu có, hàm lần ngược từ goal về start cho tới None, sau đó đảo danh sách bằng path[::-1] để thu được thứ tự start → ... → goal.

Ý nghĩa: A* và Dijkstra cập nhật parent[nxt] chỉ khi tìm được new_g nhỏ hơn best_g[nxt]. Vì vậy reconstruct_path luôn phản ánh chuỗi cha tương ứng với đường tốt nhất hiện biết tới goal tại thời điểm kết thúc.

```
def reconstruct_path(parent, goal):
    if goal not in parent:
        return None

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]

    return path[::-1]
```

parent chỉ được cập nhật khi best_g được cải thiện, vì vậy đường dựng lại luôn tương ứng với đường tốt nhất hiện biết tới goal.

4.3. Tạo graph cho grid 4 hướng

Mục đích: Hàm build_grid_graph(rows, cols, obstacles) chuyển grid thành graph kề để các thuật toán tìm kiếm có thể làm việc thống nhất. Mỗi ô hợp lệ (r, c) là một node; các ô vật cản bị loại khỏi graph.

Cách hoạt động: Với mỗi node, hàm xét bốn dịch chuyển (-1,0), (1,0), (0,-1), (0,1), tương ứng lên, xuống, trái, phải. Một neighbor chỉ được thêm nếu còn nằm trong biên grid và không thuộc obstacles. Mỗi cạnh được gán trọng số 1.0, nên chi phí của một path bằng số bước di chuyển.

Kết quả trả về: graph là dictionary hai tầng: graph[node][neighbor] = edge_cost. Ví dụ graph[(2,3)][(2,4)] = 1.0 nghĩa là từ ô (2,3) có thể đi sang (2,4) với chi phí 1.

```
def build_grid_graph(rows, cols, obstacles):
    obstacles = set(obstacles)
    graph = {}

    for r in range(rows):
        for c in range(cols):
            node = (r, c)
            if node in obstacles:
                continue

            neighbors = {}
            for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
                nr, nc = r + dr, c + dc
                if (0 <= nr < rows and 0 <= nc < cols
                        and (nr, nc) not in obstacles):
                    neighbors[(nr, nc)] = 1.0

            graph[node] = neighbors

    return graph
```

4.4. Sinh obstacle và bảo đảm grid reachable

Mục đích: Phần này có hai hàm. is_reachable(...) dùng BFS để kiểm tra xem một tập obstacle có còn ít nhất một đường từ start tới goal hay không; generate_obstacles(...) sinh obstacle theo mật độ và seed, nhưng chỉ chấp nhận grid reachable.

Hàm is_reachable: Khởi tạo queue với start và tập visited để tránh lặp. Mỗi lần lấy một ô khỏi queue, hàm kiểm tra goal rồi mở rộng bốn ô kề hợp lệ. Nếu gặp goal thì trả về True; nếu queue rỗng mà chưa gặp goal thì trả về False. BFS ở đây chỉ là phép kiểm tra tính hợp lệ của dữ liệu, không phải thuật toán cần so sánh trong thí nghiệm.

Hàm generate_obstacles: Danh sách cells chứa tất cả ô trừ start và goal. Số vật cản được tính bởi k = round(density × rows × cols). random.Random(...) tạo bộ sinh số ngẫu nhiên cục bộ để tái lập thí nghiệm. Nếu grid đầu tiên không reachable, hàm thử lại với seed biến đổi theo attempt; tối đa max_attempts lần rồi báo lỗi.

Lưu ý thực nghiệm: Với cùng density và seed, chỉ sinh obstacle một lần rồi dùng chính graph đó cho Dijkstra, A*, Weighted A* và Greedy. Như vậy mọi thuật toán được so sánh trên cùng dữ liệu đầu vào.

```
def is_reachable(rows, cols, obstacles, start, goal):
    obstacles = set(obstacles)
    if start in obstacles or goal in obstacles:
        return False

    q = deque([start])
    visited = {start}

    while q:
        r, c = q.popleft()
        if (r, c) == goal:
            return True

        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            nxt = (r + dr, c + dc)
            if (0 <= nxt[0] < rows and 0 <= nxt[1] < cols
                    and nxt not in obstacles
                    and nxt not in visited):
                visited.add(nxt)
                q.append(nxt)

    return False


def generate_obstacles(rows, cols, density, seed,
                       start, goal, max_attempts=200):
    cells = [(r, c)
             for r in range(rows)
             for c in range(cols)
             if (r, c) not in {start, goal}]

    k = int(round(density * rows * cols))

    for attempt in range(max_attempts):
        rng = random.Random(seed + attempt * 10007)
        obstacles = set(rng.sample(cells, k))

        if is_reachable(rows, cols, obstacles, start, goal):
            return obstacles

    raise RuntimeError('Không tạo được grid reachable')
```

BFS trong is_reachable chỉ dùng để kiểm tra grid có đường nối hay không; nó không thay thế thuật toán A* cần cài trong bài.

4.5. Các heuristic

Mục đích: Các hàm heuristic ước lượng chi phí còn lại từ node hiện tại tới goal. Chúng không thay đổi g(n); chúng chỉ ảnh hưởng thứ tự ưu tiên trong frontier.

h_zero: Trả về h(n)=0 cho mọi node. Khi đó f(n)=g(n)+0=g(n), nên A*-h0 tương đương Dijkstra/Uniform-Cost Search.

Manhattan: Với node n=(r,c) và goal=(r_g,c_g), h_M(n)=|r-r_g|+|c-c_g|. Trên grid chỉ đi bốn hướng và mỗi bước cost=1, Manhattan là lower bound chính xác theo hình học lưới, nên admissible và consistent.

Euclidean: h_E(n)=sqrt((r-r_g)^2+(c-c_g)^2). Khoảng cách đường thẳng không vượt quá số bước cần đi trên grid 4 hướng, nên cũng admissible. Tuy nhiên nó thường nhỏ hơn Manhattan, vì vậy định hướng tìm kiếm yếu hơn và có thể mở rộng nhiều node hơn.

Weighted Manhattan: Hàm trả về h_w(n)=w·h_M(n), mặc định w=1.5. Khi dùng trong A*, f(n)=g(n)+1.5h_M(n). Heuristic mạnh hơn nên thường ưu tiên tiến về goal nhanh hơn, nhưng do có thể overestimate nên không còn bảo đảm path tối ưu.

```
def h_zero(node, goal):
    return 0.0


def manhattan(node, goal):
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])


def euclidean(node, goal):
    return math.sqrt(
        (node[0] - goal[0])**2 +
```

```
        (node[1] - goal[1])**2
    )


def weighted_manhattan(node, goal, w=1.5):
    return w * manhattan(node, goal)
```

4.6. Dijkstra làm baseline tối ưu

Dijkstra là A* với h=0. Ta vẫn dùng best_g và bỏ stale entry để cấu trúc cài đặt nhất quán với A*.

Mục đích: Hàm dijkstra(graph, start, goal) được dùng làm baseline để lấy cost_opt cho từng grid. Với trọng số cạnh không âm, Dijkstra trả về chi phí đường đi tối ưu.

Các biến chính: pq là priority queue chứa (g, tie_breaker, node); best_g lưu chi phí nhỏ nhất đã biết tới mỗi node; parent dùng dựng path; expanded đếm số entry hợp lệ được pop và xử lý; max_frontier ghi kích thước lớn nhất của pq trong quá trình chạy. order = count() tạo tie_breaker để heap không phải so sánh trực tiếp node khi g bằng nhau.

Cách hoạt động: Mỗi lần pop node có g nhỏ nhất. Nếu g của entry lớn hơn best_g[node], đó là stale entry và bị bỏ qua. Với mỗi neighbor nxt, tính new_g=g+w. Nếu new_g < best_g[nxt], thực hiện relaxation: cập nhật best_g, parent và push entry mới vào pq. Khi goal được pop bằng entry hợp lệ, chi phí g chính là cost_opt.

Kết quả trả về: Một bộ bốn giá trị (path, cost, expanded, max_frontier). Nếu không có đường đi, path=None và cost=∞. Trong Lab, grid đã được kiểm tra reachable nên trường hợp thất bại chủ yếu dùng để bảo vệ hàm.

```
def dijkstra(graph, start, goal):
    order = count()
    pq = [(0.0, next(order), start)]
    best_g = {start: 0.0}
    parent = {start: None}
    expanded = 0
    max_frontier = 1

    while pq:
        max_frontier = max(max_frontier, len(pq))
        g, _, node = heapq.heappop(pq)

        if g > best_g.get(node, float('inf')):
            continue

        expanded += 1
        if node == goal:
            return reconstruct_path(parent, goal), g, expanded, max_frontier

        for nxt, w in graph.get(node, {}).items():
            new_g = g + w
            if new_g < best_g.get(nxt, float('inf')):
                best_g[nxt] = new_g
                parent[nxt] = node
                heapq.heappush(pq, (new_g, next(order), nxt))

    return None, float('inf'), expanded, max_frontier
```

4.7. Cài đặt A* Search

Mục đích: Hàm astar(graph, h, start, goal) cài A* tổng quát, trong đó h là một hàm heuristic được truyền từ bên ngoài. Nhờ vậy cùng một khung A* có thể chạy với h=0, Manhattan, Euclidean hoặc Weighted Manhattan.

Priority queue: Mỗi entry có dạng (f, tie_breaker, g, node), trong đó f=g+h. Python heapq luôn pop tuple nhỏ nhất theo thứ tự từ trái sang phải; tie_breaker giúp xử lý trường hợp nhiều node có cùng f mà không phụ thuộc cách so sánh tọa độ node.

Relaxation và stale entry: Khi mở rộng node, với mỗi nxt tính new_g=g+w. Chỉ khi new_g < best_g[nxt] mới cập nhật best_g[nxt], parent[nxt] và push entry mới với new_f=new_g+h(nxt). Entry cũ vẫn có thể còn trong heap; khi được pop sau này, điều kiện g > best_g[node] nhận biết và bỏ qua nó.

Điều kiện dừng: Không dừng khi goal vừa được sinh ra. A* chỉ dừng khi goal được pop khỏi priority queue bằng một entry hợp lệ. Với heuristic admissible/consistent trong điều kiện của Lab, cách dừng này cho phép A* trả về đường tối ưu; với Weighted A*, path có thể suboptimal vì heuristic đã được nhân trọng số lớn hơn 1.

Kết quả trả về: Giống Dijkstra: (path, cost, expanded, max_frontier). Nhờ cùng cấu trúc trả về, các thuật toán có thể được benchmark và đưa vào cùng bảng thống kê.

```
def astar(graph, h, start, goal):
    order = count()

    # Entry: (f, tie_breaker, g, node)
    pq = [(h(start), next(order), 0.0, start)]
    best_g = {start: 0.0}
    parent = {start: None}

    expanded = 0
    max_frontier = 1

    while pq:
        max_frontier = max(max_frontier, len(pq))
        f, _, g, node = heapq.heappop(pq)

        # Entry cũ: đã có đường tốt hơn tới node
        if g > best_g.get(node, float('inf')):
            continue

        expanded += 1

        # Chỉ dừng khi goal được POP với entry hợp lệ
        if node == goal:
            path = reconstruct_path(parent, goal)
            return path, g, expanded, max_frontier

        for nxt, w in graph.get(node, {}).items():
            new_g = g + w

            # Relaxation
            if new_g < best_g.get(nxt, float('inf')):
                best_g[nxt] = new_g
                parent[nxt] = node
                new_f = new_g + h(nxt)
                heapq.heappush(
                    pq,
                    (new_f, next(order), new_g, nxt)
                )

    return None, float('inf'), expanded, max_frontier
```

Lưu ý: Không thay best_g bằng visited đơn giản. Một node đã từng được phát hiện vẫn có thể nhận new_g nhỏ hơn; khi đó phải cập nhật parent và push entry mới.

4.8. Greedy-Manhattan để đối chứng

Greedy chỉ xếp thứ tự theo h(n), không dùng g(n). Với grid unit-cost, cost của path trả về bằng len(path)-1.

Mục đích: Hàm greedy_best_first(graph, h, start, goal) là baseline định hướng để làm rõ khác biệt giữa Greedy và A*. Greedy chỉ dùng h(n) để xếp thứ tự, tương đương f(n)=h(n), còn A* dùng f(n)=g(n)+h(n).

Cách hoạt động: Priority queue lưu (h, tie_breaker, node). visited được đánh dấu ngay khi node được phát hiện để tránh đưa cùng node vào queue nhiều lần. parent ghi node cha lần đầu; khi goal được pop, reconstruct_path dựng lại đường đi.

Chi phí và giới hạn: Do Lab dùng grid unit-cost, cost có thể tính bằng len(path)-1. Cách tính này chỉ đúng khi mọi cạnh có cost=1. Greedy thường mở ít node và đi nhanh về phía goal, nhưng vì bỏ qua g(n) nên không bảo đảm tìm path tối ưu và có thể có optimality gap dương.

```
def greedy_best_first(graph, h, start, goal):
    order = count()
    pq = [(h(start), next(order), start)]
    visited = {start}
    parent = {start: None}

    expanded = 0
    max_frontier = 1

    while pq:
        max_frontier = max(max_frontier, len(pq))
        _, _, node = heapq.heappop(pq)
        expanded += 1
```

```
        if node == goal:
            path = reconstruct_path(parent, goal)
            cost = len(path) - 1
            return path, cost, expanded, max_frontier

        for nxt in graph.get(node, {}):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = node
                heapq.heappush(pq, (h(nxt), next(order), nxt))

    return None, float('inf'), expanded, max_frontier
```

4.9. Benchmark và optimality gap

Hàm benchmark: Nhận search_fn là một hàm không tham số, chạy lại cùng phép tìm kiếm repeats lần (mặc định 5), đo thời gian bằng perf_counter(), đổi sang millisecond và lấy trung bình. result lưu kết quả của lần chạy cuối; về logic, các lần phải cho cùng path/cost vì graph và thuật toán không thay đổi.

Hàm optimality_gap: So sánh chi phí của thuật toán với chuẩn Dijkstra theo công thức Gap(%) = 100 × (cost - cost_opt) / cost_opt. Gap=0% nghĩa là đạt chi phí tối ưu; Gap>0% cho biết nghiệm dài/đắt hơn chuẩn bao nhiêu phần trăm. Nếu cost_opt=∞, hàm trả NaN vì không có nghiệm tối ưu hữu hạn để làm mẫu số.

Ý nghĩa khi đọc kết quả: Không nên đánh giá thuật toán chỉ bằng runtime. Cần đọc đồng thời Gap %, Expanded và Max frontier để thấy sự đánh đổi giữa chất lượng nghiệm, lượng công việc tìm kiếm và nhu cầu bộ nhớ.

```
def benchmark(search_fn, repeats=5):
    times = []
    result = None

    for _ in range(repeats):
        t0 = perf_counter()
        result = search_fn()
        times.append((perf_counter() - t0) * 1000)

    return result, sum(times) / len(times)


def optimality_gap(cost, cost_opt):
    if cost_opt == float('inf'):
        return float('nan')
    return 100.0 * (cost - cost_opt) / cost_opt
```

4.10. Chạy thử một grid với nhiều cấu hình

Mục đích: Khối lệnh này kiểm tra toàn bộ pipeline trên một grid trước khi chạy thí nghiệm lớn. Grid 20×20 được sinh ở mật độ 20% với seed=42; sau đó graph được dựng một lần và dùng chung cho tất cả cấu hình.

Trình tự thực hiện: Dijkstra chạy trước để lấy cost_opt. Danh sách configs chứa tên cấu hình và hàm heuristic tương ứng. Vòng lặp gọi astar(...) cho từng heuristic, benchmark runtime, tính Gap % so với cost_opt rồi thêm các chỉ số vào rows_result. Cuối cùng pd.DataFrame(rows_result) hiển thị bảng kết quả để kiểm tra nhanh.

Điểm cần kiểm tra: A*-h0 phải có cost bằng Dijkstra. A*-Manhattan và A*-Euclidean trên grid 4 hướng unit-cost cũng được kỳ vọng Gap≈0 nếu cài đặt đúng. Weighted A* có thể có Gap>0 nhưng thường giảm Expanded.

```
rows = cols = 20
start = (0, 0)
goal = (19, 19)

obstacles = generate_obstacles(
    rows, cols,
    density=0.20,
    seed=42,
    start=start,
    goal=goal
)
graph = build_grid_graph(rows, cols, obstacles)

# Dijkstra lấy cost tối ưu
(path_opt, cost_opt, exp_opt, front_opt), t_opt = benchmark(
    lambda: dijkstra(graph, start, goal)
)
```

```
configs = [
    ('A*-h0', lambda n: h_zero(n, goal)),
    ('A*-Manhattan', lambda n: manhattan(n, goal)),
    ('A*-Euclidean', lambda n: euclidean(n, goal)),
    ('Weighted A* w=1.5', lambda n: weighted_manhattan(n, goal, 1.5)),
]

rows_result = []
for name, h in configs:
    (path, cost, expanded, max_frontier), avg_ms = benchmark(
        lambda h=h: astar(graph, h, start, goal)
    )

    rows_result.append({
        'Algorithm': name,
        'Cost': cost,
        'Gap %': optimality_gap(cost, cost_opt),
        'Expanded': expanded,
        'Max frontier': max_frontier,
        'Runtime (ms)': avg_ms
    })

pd.DataFrame(rows_result)
```

4.11. Chạy 3 mật độ × 10 seed

Mục đích: Đây là vòng thí nghiệm chính. Ba mật độ 10%, 20%, 30% được kết hợp với 10 seed cố định, tạo 30 grid. Trên mỗi grid, chạy Dijkstra và năm cấu hình còn lại, vì vậy bảng dữ liệu chi tiết có 30 × 6 = 180 dòng.

Cách bảo đảm công bằng: Với mỗi cặp (density, seed), obstacles và graph chỉ được sinh một lần. Dijkstra lấy cost_opt trên chính graph đó; sau đó A*-h0, A*-Manhattan, A*-Euclidean, Weighted A* và Greedy dùng lại graph, start và goal giống hệt nhau.

Dữ liệu lưu: Mỗi dòng trong all_rows chứa Density, Seed, Algorithm, Cost, Gap %, Expanded, Max frontier và Runtime (ms). Sau khi vòng lặp kết thúc, results = pd.DataFrame(all_rows) tạo bảng dữ liệu cấp lần chạy; đây là nguồn để tính các mean ở mục 4.12 và bảng 5.1.

```
densities = [0.10, 0.20, 0.30]
seeds = [11, 22, 33, 44, 55, 66, 77, 88, 99, 110]
all_rows = []

for density in densities:
    for seed in seeds:
        start, goal = (0, 0), (19, 19)
        obstacles = generate_obstacles(
            20, 20, density, seed,
            start=start, goal=goal
        )
        graph = build_grid_graph(20, 20, obstacles)

        # Baseline Dijkstra
        (p0, cost_opt, e0, m0), t0 = benchmark(
            lambda: dijkstra(graph, start, goal)
        )
        all_rows.append({
            'Density': density,
            'Seed': seed,
            'Algorithm': 'Dijkstra',
            'Cost': cost_opt,
            'Gap %': 0.0,
            'Expanded': e0,
            'Max frontier': m0,
            'Runtime (ms)': t0
        })

        configs = [
            ('A*-h0', lambda n, g=goal: h_zero(n, g)),
            ('A*-Manhattan', lambda n, g=goal: manhattan(n, g)),
            ('A*-Euclidean', lambda n, g=goal: euclidean(n, g)),
            ('Weighted A* w=1.5', lambda n, g=goal: 1.5 * manhattan(n, g)),
            ('Greedy-Manhattan', lambda n, g=goal: manhattan(n, g)),
        ]
```

```
    for name, h in configs:
        if name == 'Greedy-Manhattan':
            (path, cost, ex, mf), ms = benchmark(
                lambda h=h: greedy_best_first(graph, h, start, goal)
            )
        else:
            (path, cost, ex, mf), ms = benchmark(
                lambda h=h: astar(graph, h, start, goal)
            )

        all_rows.append({
            'Density': density,
            'Seed': seed,
            'Algorithm': name,
            'Cost': cost,
            'Gap %': optimality_gap(cost, cost_opt),
            'Expanded': ex,
            'Max frontier': mf,
            'Runtime (ms)': ms
        })

results = pd.DataFrame(all_rows)
results.head()
```

4.12. Tổng hợp kết quả theo mật độ

Mục đích: Khối này chuyển bảng chi tiết results thành bảng tóm tắt theo từng cặp Density–Algorithm để có đúng dạng cần báo cáo ở mục 5.1.

Cách hoạt động: groupby([Density, Algorithm]) gom các dòng có cùng mật độ và thuật toán. agg(...: mean) lấy trung bình Cost, Gap %, Expanded, Max frontier và Runtime trên 10 seed. reset_index() đưa hai khóa nhóm trở lại thành cột bình thường để dễ hiển thị hoặc xuất file.

Ý nghĩa: Mỗi dòng của summary là hiệu năng trung bình của một thuật toán tại một mức obstacle. Do đó Mean Cost, Mean Gap %, Mean Expanded, Mean Max frontier và Mean Runtime trong bảng 5.1 có thể lấy trực tiếp từ summary.

```
summary = (
    results
    .groupby(['Density', 'Algorithm'])
    .agg({
        'Cost': 'mean',
        'Gap %': 'mean',
        'Expanded': 'mean',
        'Max frontier': 'mean',
        'Runtime (ms)': 'mean'
    })
    .reset_index()
)

summary
```

4.13. Trực quan hóa grid và đường đi

Mục đích: Hàm draw_grid(...) vẽ obstacle, path, start và goal để kiểm tra trực quan đường đi mà thuật toán tìm được. Đây là bước quan trọng để phát hiện lỗi path dù các chỉ số số học có vẻ hợp lý.

Cách vẽ: Mỗi obstacle được biểu diễn bằng một Rectangle tại tọa độ cột–hàng. Với path, tọa độ được cộng 0.5 để đường đi qua tâm mỗi ô. Start dùng marker hình tròn, goal dùng marker ngôi sao. plt.ylim(rows, 0) đảo trục y để hàng 0 nằm phía trên, giống cách nhìn ma trận.

Đoạn gọi cuối: h được gán là Manhattan, sau đó astar(...) tạo path và cost trên graph hiện có. draw_grid(...) nhận path cùng start, goal và title để hiển thị. Khi làm bài bắt buộc, sinh viên nên dùng cùng một grid để vẽ ít nhất hai path, ví dụ A*-Manhattan và Weighted A* hoặc Greedy, nhằm quan sát sự khác biệt về tuyến đường.

```
def draw_grid(rows, cols, obstacles, path=None,
              start=None, goal=None, title='Route Planning'):
    plt.figure(figsize=(7, 7))
    ax = plt.gca()

    for r, c in obstacles:
        ax.add_patch(plt.Rectangle((c, r), 1, 1, fill=True))

    if path:
```

```
        xs = [c + 0.5 for r, c in path]
        ys = [r + 0.5 for r, c in path]
        plt.plot(xs, ys, linewidth=3)

    if start:
        plt.scatter(start[1] + 0.5, start[0] + 0.5,
                    s=120, marker='o', label='Start')
    if goal:
        plt.scatter(goal[1] + 0.5, goal[0] + 0.5,
                    s=160, marker='*', label='Goal')

    plt.xlim(0, cols)
    plt.ylim(rows, 0)
    plt.xticks(range(cols + 1))
    plt.yticks(range(rows + 1))
    plt.grid(True, linewidth=0.4)
    plt.legend()
    plt.title(title)
    plt.show()

h = lambda n: manhattan(n, goal)
path, cost, _, _ = astar(graph, h, start, goal)
draw_grid(
    rows, cols, obstacles,
    path=path,
    start=start,
    goal=goal,
    title=f'A* Manhattan – cost={cost}'
)
```

5. Bài tập bắt buộc

1. Sinh grid 20×20 ở ba mật độ 10%, 20%, 30%; trực quan hóa ít nhất một grid ở mỗi mức.
2. Cài A* bằng priority queue, best_g và parent. Không dùng visited đơn giản để khóa node ngay lần đầu phát hiện.
3. Chạy Dijkstra, A*-h0, A*-Manhattan, A*-Euclidean, Weighted A* w=1.5 và Greedy-Manhattan trên cùng 10 seed cố định cho mỗi mật độ.
4. Dùng Dijkstra làm chuẩn cost_opt cho từng seed; tính optimality gap của mọi cấu hình còn lại.
5. Kiểm tra tính hợp lệ của path và thống kê Cost, Gap %, Expanded, Max frontier, Runtime cho từng lần chạy.
6. Vẽ ít nhất hai path trên cùng một grid: A*-Manhattan và Weighted A* hoặc Greedy-Manhattan; nhận xét sự khác nhau về đường đi.
7. Viết kết luận dựa trên số liệu: cấu hình nào giữ optimality tốt, cấu hình nào giảm expanded/runtime, và đánh đổi của Weighted A*/Greedy là gì.

5.1. Mẫu bảng kết quả và cách tính

Mỗi hàng trong bảng dưới đây là kết quả trung bình trên 10 seed tại cùng một mật độ obstacle. Tất cả thuật toán trong cùng một density phải chạy trên đúng cùng 10 grid. Dijkstra được dùng làm chuẩn nên Mean Gap của Dijkstra bằng 0%.

Obstacle	Thuật toán / Heuristic	Mean Cost	Mean Gap %	Mean Expanded	Mean Max frontier	Mean Runtime ms
10%	Dijkstra					
10%	A*-h0					
10%	A*-Manhattan					
10%	A*-Euclidean					
10%	Weighted A* w=1.5					
10%	Greedy-Manhattan					
20%	Dijkstra					
20%	A*-h0					
20%	A*-Manhattan					

Obstacle	Thuật toán / Heuristic	Mean Cost	Mean Gap %	Mean Expanded	Mean Max frontier	Mean Runtime ms
20%	A*-Euclidean					
20%	Weighted A* w=1.5					
20%	Greedy-Manhattan					
30%	Dijkstra					
30%	A*-h0					
30%	A*-Manhattan					
30%	A*-Euclidean					
30%	Weighted A* w=1.5					
30%	Greedy-Manhattan					

Cách hiểu và điền từng cột:

- Obstacle: tỷ lệ ô bị chặn trong grid. Start và Goal không được là obstacle.
- Mean Cost: trung bình chi phí đường đi trên 10 seed. Với grid unit-cost, cost = số bước = len(path)-1.

$$\text{Mean Cost} = (\text{cost}_1 + \text{cost}_2 + \cdots + \text{cost}_{10}) / 10$$

- Mean Gap %: trước hết tính gap riêng cho từng seed so với Dijkstra trên chính grid đó, sau đó mới lấy trung bình. Không lấy Mean Cost rồi mới tính một gap chung.

$$\text{Gap}_i(\%) = ((\text{cost}_i - \text{cost_opt},_i) / \text{cost_opt},_i) \times 100$$

$$\text{Mean Gap}(\%) = (\text{Gap}_1 + \text{Gap}_2 + \cdots + \text{Gap}_{10}) / 10$$

- Mean Expanded: trung bình số node non-stale được pop và xử lý. Chỉ số càng nhỏ thì lượng tìm kiếm thường càng ít.
- Mean Max frontier: trung bình kích thước lớn nhất của OPEN trong mỗi lần chạy; phản ánh gần đúng nhu cầu bộ nhớ.
- Mean Runtime ms: trung bình thời gian chạy thuật toán, đơn vị millisecond. Không tính thời gian tạo grid hoặc vẽ hình.

Ví dụ minh họa cách tính gap: nếu trên một seed Dijkstra cho cost_opt=40 và Weighted A* cho cost=42 thì:

$$\text{Gap} = ((42 - 40) / 40) \times 100 = 5\%$$

Nếu A*-Manhattan cũng tìm cost=40 thì Gap=0%. Với implementation đúng trên grid 4 hướng unit-cost, A*-Manhattan và A*-Euclidean được kỳ vọng có gap bằng 0%; Weighted A* và Greedy có thể có gap dương.

5.2. Cách đọc kết quả và viết kết luận

Không nên kết luận chỉ dựa trên runtime. Cần đọc đồng thời chất lượng nghiệm và chi phí tìm kiếm:

- Optimality: ưu tiên Mean Gap %; 0% nghĩa là giữ được đường tối ưu so với Dijkstra.
- Search effort: Mean Expanded càng thấp cho thấy heuristic định hướng tìm kiếm tốt hơn.
- Memory: Mean Max frontier càng thấp thì OPEN thường cần ít bộ nhớ hơn.
- Runtime: Mean Runtime thấp hơn là tốt, nhưng phải xem có đánh đổi bằng Gap cao hơn hay không.

Một kết luận tốt cần có số liệu cụ thể, ví dụ: "A*-Manhattan giữ Mean Gap=0% nhưng giảm Mean Expanded từ … xuống … so với Dijkstra; Weighted A* tiếp tục giảm Expanded còn … nhưng Mean Gap tăng lên …%". Không viết chung chung "thuật toán A nhanh hơn" nếu không dẫn số liệu.

6. Ứng dụng thực tế và xu hướng

6.1. Ứng dụng thực tế

- Robot navigation và autonomous systems: tìm đường trên occupancy grid hoặc graph môi trường.
- Game AI: pathfinding trên tile map, waypoint graph hoặc navigation mesh.
- Logistics và route planning: tìm tuyến theo khoảng cách, thời gian, năng lượng hoặc chi phí.

- Puzzle và planning: tìm chuỗi hành động có tổng cost nhỏ nhất khi có heuristic ước lượng khoảng cách tới goal.

6.2. Xu hướng mở rộng

- Weighted A*: tăng trọng số heuristic để ưu tiên tốc độ, chấp nhận khả năng nghiệm suboptimal.
- Anytime search (ví dụ ARA*): nhanh chóng sinh nghiệm ban đầu rồi cải thiện dần nếu còn thời gian tính toán.
- D* / D* Lite: tái lập kế hoạch khi môi trường hoặc chi phí cạnh thay đổi, phù hợp robot trong môi trường động.
- Learned heuristic / learning-guided search: dùng mô hình học máy để học heuristic hoặc thứ tự ưu tiên trạng thái.

6.3. Câu hỏi tự đánh giá

- Vì sao A* cần best_g thay vì chỉ dùng visited như BFS?
- Khi h(n)=0, A* tương đương thuật toán nào? Vì sao?
- Vì sao Manhattan là admissible và consistent trên grid 4 hướng unit-cost?
- A*-Euclidean có tối ưu trên grid 4 hướng không? Vì sao nó thường mở rộng nhiều node hơn Manhattan?
- Tại sao 1.5×Manhattan có thể làm giảm expanded nodes nhưng tạo optimality gap?
- Trong ví dụ S–A–C–G, vì sao A* không dừng ngay khi G vừa được sinh ra với g=12?
- Greedy và A* khác nhau ở thành phần nào của hàm ưu tiên?
- Nếu môi trường thay đổi liên tục khi robot đang di chuyển, A* chuẩn có hạn chế gì?